

Vision Day1 hw1 AI보고서

로봇 21기 예비단원 최윤서

1. 사용 이유

교육 자료에 있는 VideoCapture cap(0); .. 부분을 참고하여 작성했는데 구글링을 통해 노트북에 내제된 웹캠을 불러오는 방법을 많이 찾아봤는데 마땅한 예제를 찾지 못했고

https://diyver.tistory.com/56#google_vignette

이 링크에서 구현 예제를 찾았지만 카메라 프레임을 불러올 수 없다는 결과가 나왔습니다. gpt에게 ubuntu 환경에서 열 수 없는 것인지 코드가 잘못된 것인지 판단하기 위해 우선 코드에서 수정해야할 부분이 있냐고 질문했습니다.

2. 대화 내용

"https://diyver.tistory.com/56#google_vignette 여기서 예제 코드 참고해서 카메라 불러오려고했는데 자꾸 프레임을 불러올 수 없대 어떻게 수정해야돼?"

처음 참고한 예제는 VideoCapture cap(0) 으로 카메라를 열고 cap >> img 로 프레임을 가져오는 Windows 환경을 전제로 한 단순 OpenCV 예제입니다. 해당 글에서도 VideoCapture cap(0) → isOpened() → cap >> img 순서로 카메라를 읽습니다. [3D 모델링 프런티어 ...](#)

그런데 현재 환경은 Ubuntu + ROS 2 + OpenCV + 노트북 내장 IPU6 카메라이므로, 예제 코드를 그대로 사용하는 것에서 문제가 발생할 수 있습니다. 수정 과정은 다음과 같이 진행하면 됩니다.

1단계 — 기본 OpenCV 코드 확인

처음에는 예제처럼 다음 구조로 카메라를 열려고 합니다.

```
1  C++
2
3  cv::VideoCapture cap(0);
4
5  if (!cap.isOpened())
6  {
7      std::cout << "Can't open the camera" << std::endl;
8      return -1;
9  }
10
11  cv::Mat frame;
12
13  while (true)
14  {
15      cap >> frame;
16
17      if (frame.empty())
18      {
19          std::cout << "Can't receive frame" << std::endl;
20          break;
21      }
22  }
```

예제에서도 cap 자체가 영상 데이터를 담는 것이 아니라 카메라를 준비하고, 실제 영상 프레임은 cap >> img 를 통해 cv::Mat 에 저장한다고 설명합니다. [3D 모델링 프런티어 ...](#)

그런데 여기서 frame.empty() 가 발생하거나 카메라를 제대로 열지 못하는 문제가 발생합니다.

2단계 — Ubuntu에서는 Windows용 CAP_DSHOW를 사용할 수 없음

원래 참고한 글의 해결 방법에는 다음 코드가 있습니다.

```
</> C++  
  
VideoCapture cap(0, CAP_DSHOW);
```

하지만 CAP_DSHOW는 Windows의 DirectShow 백엔드입니다. 현재 사용하는 Ubuntu에서는 이 방법을 그대로 적용할 수 없습니다.

따라서 Ubuntu에서는 V4L2를 사용하도록 변경합니다.

```
</> C++  
  
cap_.open(0, cv::CAP_V4L2);
```

현재 작성한 코드의

```
</> C++  
  
if (is_index) {  
    cap_.open(std::stoi(camera_source_), cv::CAP_V4L2);  
}
```

가 바로 이 부분을 해결한 것입니다.

3단계 — 해상도 설정

원래 예제에서는 다음과 같이 해상도를 설정합니다. 

```
</> C++  
  
cap.set(CV_CAP_PROP_FRAME_WIDTH, 1920);  
cap.set(CV_CAP_PROP_FRAME_HEIGHT, 1080);
```

현재 OpenCV에서는 다음처럼 변경합니다.

```
</> C++  
  
cap_.set(cv::CAP_PROP_FRAME_WIDTH, frame_width_);  
cap_.set(cv::CAP_PROP_FRAME_HEIGHT, frame_height_);
```

그리고 실제 장치가 해당 해상도를 받아들였는지 확인하도록 추가합니다.

```
</> C++  
  
const int actual_w =  
    static_cast<int>(cap_.get(cv::CAP_PROP_FRAME_WIDTH));  
  
const int actual_h =  
    static_cast<int>(cap_.get(cv::CAP_PROP_FRAME_HEIGHT));
```

즉,

```
요청  
1920 × 1080  
    ↓  
카메라 장치  
    ↓  
실제로 지원하는 해상도 확인
```

하도록 수정합니다.

4단계 — 카메라가 열렸어도 read() 가 실패하는 경우 처리

단순 예제에서는

```
</> C++  
cap >> img;
```

만 사용하지만, 현재 코드는 다음처럼 명시적으로 프레임 획득 성공 여부를 검사합니다.

```
</> C++  
  
if (!cap_.read(frame) || frame.empty()) {  
    RCLCPP_WARN(  
        this->get_logger(),  
        "프레임을 읽지 못했습니다.");  
    return;  
}
```

따라서

```
카메라 열기  
↓  
cap_.isOpened()  
↓  
프레임 읽기  
↓  
cap_.read(frame)  
↓  
frame.empty() 검사
```

순서로 오류를 확인할 수 있게 됩니다.

5단계 — 단독 OpenCV 프로그램이 아니라 ROS2 Camera Node로 변경

처음 예제는 단순히

```
</> C++  
imshow("camera img", img);
```

로 화면에 표시합니다. [3D모델링 프린팅 ...](#)

하지만 이번 과제에서는 **Camera Node**와 **UI Node**를 분리해야 하므로 imshow() 를 사용하지 않고 카메라에서 얻은 cv::Mat 을 ROS Image 메시지로 변환해서 publish합니다.

```
</> C++  
  
std_msgs::msg::Header header;  
header.stamp = this->now();  
header.frame_id = "camera_frame";  
  
const auto msg =  
    cv_bridge::CvImage(  
        header,  
        "bgr8",  
        frame  
    ).toImageMsg();  
  
image_pub_.publish(msg);
```

즉 기존의

```
Camera
↓
cv::Mat
↓
imshow()
```



를

```
Camera
↓
cv::Mat
↓
cv_bridge
↓
sensor_msgs/Image
↓
image_raw
```



로 변경합니다.

✦ Upgrade ↗

6단계 — while 문 대신 ROS2 Timer Callback 사용

기존 예제는

```
</> C++
```



구조입니다.

하지만 ROS2 Node에서는 무한 루프를 직접 돌리는 대신 Timer를 사용합니다.

현재 코드의

```
</> C++

timer_ = this->create_wall_timer(
    std::chrono::duration_cast<std::chrono::milliseconds>(period),
    std::bind(&CameraNode::timerCallback, this));
```



가 이 역할을 합니다.

그리고 실제 카메라 프레임 획득부터 publish까지를

```

</> C++
void CameraNode::timerCallback()
{
    cv::Mat frame;

    if (use_synthetic_) {
        frame = generateSyntheticFrame();
    }
    else {
        if (!cap_.read(frame) || frame.empty()) {
            RCLCPP_WARN(
                this->get_logger(),
                "프레임을 읽지 못했습니다.");
            return;
        }
    }

    // cv::Mat → ROS Image → publish
}

```

안에서 처리합니다.

7단계 — 실제 카메라가 안 될 때 합성 영상으로 분리

여기서 현재 코드에 추가된 중요한 부분이

```

</> C++
bool use_synthetic_;

```

입니다.

```

</> YAML
use_synthetic: false

```

이면 실제 카메라를 사용하고,

```

</> YAML
use_synthetic: true

```

이면 실제 카메라 대신

```

</> C++
cv::Mat CameraNode::generateSyntheticFrame()

```

으로 테스트 영상을 생성합니다.

따라서 카메라 장치 문제와 ROS/Qt 통신 문제를 분리해서 확인할 수 있습니다.

3. 느낀 점

총 문제점은 2개였습니다. ubuntu에서 내장 카메라가 일반 V4L2 카메라라처럼 동작하지 않는 문제와 OpenCV에서 카메라를 여는 방식도 ubuntu 환경에 맞춰야했습니다. 다른 단원들의 얘기를 들어봤을 때 이런 이슈를 겪은 단원은 없었고 실제로 모든 노트북에서 발생하는 문제는 아니라고합니다. 따라서 AI에게 질문했고 구글링으로도 찾아보기 힘든 정보나 노트북 하드웨어 관련된 문제는 AI를 통해 해결해보고 터미널에서 디버깅해보는 것이 빠르다는 것을 느낄 수 있었습니다.